

# Le Classi e gli Oggetti

GSTRUMSVIL - Guida allo sviluppo in BC - © SISTEMI S.p.A.

## Introduzione allo sviluppo OOP

Gli ambienti di sviluppo che permettono di programmare ad oggetti (Object Oriented Programming, da cui l'acronimo OOP che utilizzeremo in seguito) consentono la definizione di oggetti software in grado di interagire gli uni con gli altri.

La programmazione ad oggetti è senza dubbio il modello più diffuso ed utilizzato per via degli innumerevoli vantaggi che ne derivano. Questo paradigma ha reso quindi superate le precedenti metodologie strutturate e procedurali.

Anche nell'ambiente di sviluppo Sistemi questa metodologia si affianca alla precedente (quella procedurale) introducendo concetti già previsti in altri ambienti di sviluppo, modellandoli al meglio rispetto alle specificità dell'ambiente proprietario Sistemi.

Nell'approccio procedurale possiamo distinguere due mondi diversi e separati: il mondo dei dati e il mondo del codice.

Il mondo dei dati è definito da variabili di diverso tipo e scollegate tra loro, mentre il mondo del codice è definito da funzioni che consentono il riutilizzo del codice. Le funzioni sono in grado di usare i dati, ma non viceversa.

L'approccio della OOP suggerisce un modo di pensare completamente diverso: i dati e il codice sono racchiusi nello stesso mondo, contraddistinto dalla definizione di *classi*.

## Che differenza c'è tra classi e oggetti

Una classe è un modello che consente di creare oggetti aventi attributi e metodi che saranno condivisi da tutti gli oggetti creati. Un oggetto è di fatto l'istanza di una classe.

Un *oggetto* prevede un insieme di tratti (*proprietà*) ed è in grado di eseguire un insieme di attività (*metodi*). Possono interagire tra di loro scambiando dati e/o attivando i loro metodi: non c'è più un confine tra dato e codice. Essi convivono all'interno di un oggetto.

La fase di progettazione iniziale dovrà dare quindi risposta alle domande: "*Quali proprietà sono necessarie a compiere le azioni da svolgere?*", "*Cosa si vuole che sia in grado di fare?*".

## Le classi in BC

A differenza di quanto avviene in altri linguaggi di programmazione, nell'ambiente di sviluppo Sistemi, la definizione delle classi avviene a design-time. È quindi AGIS il punto in cui vengono definite le classi identificate con lo specifico tipo di oggetto "40: Classe".

A livello di sorgente viene invece definito l'oggetto da utilizzare riferito ad una specifica classe.

Diremo quindi che un *oggetto* fa riferimento ad una specifica *classe* e costituisce un'**istanza della classe** stessa. All'interno dello stesso sorgente è possibile creare due o più istanze facenti riferimento alla stessa classe, ma ciò non significa che tutte queste istanze eseguano le stesse azioni o che siano correlate tra di loro.

## Definizione di una classe

Usiamo l'entità Cliente come esempio di definizione della nostra classe. Un Cliente può essere descritto con le seguenti caratteristiche:

- un identificativo univoco: CodCliente;
- una denominazione: RagioneSociale;
- una informazione che attesti che abbia la partita IVA: TitPIVA
- una Partita IVA che identifica il Cliente per il pagamento dell'IVA: PIVA;
- un elenco di indirizzi (fatturazione, spedizione, sede legale, ecc) che costituisce a tutti gli effetti ad un'altra entità che chiameremo Indirizzi.

La classe degli Indirizzi sarà quindi costituita da:

- un identificativo univoco: NumProgressivo;
- l'indirizzo comprensivo del numero civico (via, corso, viale, ecc...): Indirizzo;
- codice di avviamento postale: CAP;
- il comune: Comune;
- Provincia: Provincia.

Vediamo come creare le due classi su AGIS.

L'anagrafica è simile a:

Aggiunta dati oggetto

### Classi

Dati generali

Nome oggetto **1**

DFX di riferimento **2**

Descrizione

Descrizione estesa

Categoria

Data validita' da

☐ Classe statica

**3**

OK (Invio) Abbandona (F1) Copia da prodotto (F3)

Introdurre il nome dell'oggetto AGIS CALDERA GG/CB/GW

Una volta definito il "Nome oggetto", la descrizione è inserita in un contenitore di classi, possiamo andare a creare fisicamente la nostra classe.

Per il momento ignoriamo le altre informazioni, le vedremo più avanti.

Come per il Tipo Dato, anche la classe deve essere inserita all'interno di un contenitore di classi identificato con il tipo oggetto 41. L'azione di link del contenitore consente la generazione automatica di codice, a partire dall'anagrafica delle classi relazionati, che da origine ad una DLL necessaria all'esecuzione dei programmi che fanno uso delle classi.

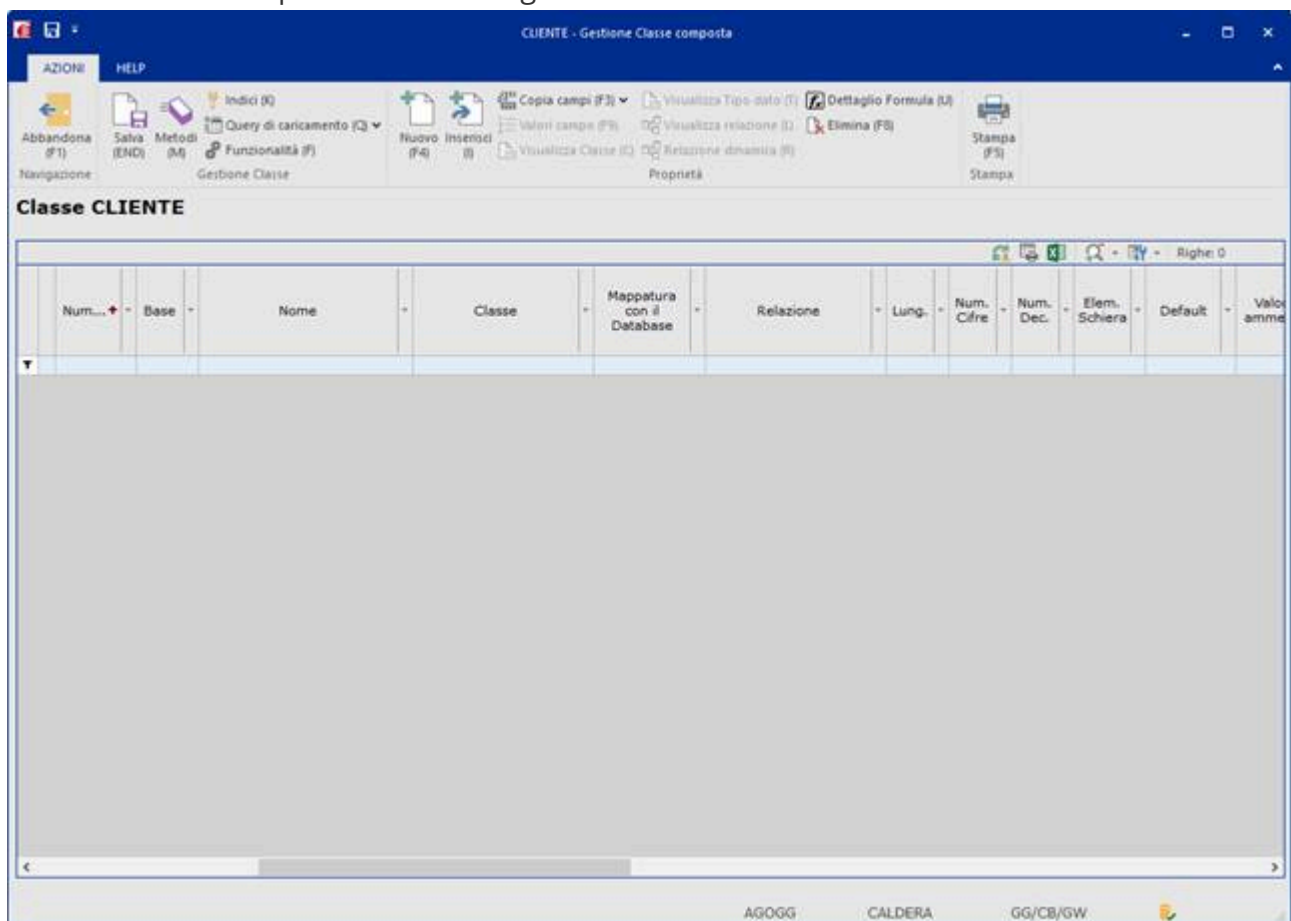


Come linea guida è preferibile battezzare i contenitori raggruppando le classi per ambito applicativo di riferimento, stesso criterio utilizzato per la definizione delle librerie.

Lo schema di riferimento è: <Sigla prodotto>\_<ambito applicativo>\_CL.

Per maggiori informazioni riguardo la nomenclatura degli oggetti si può fare riferimento alla scheda "Nomenclatura oggetti AGIS" degli *Standard di sviluppo*.

La gestione della classe si presenta come segue:



Tramite il bottone "Nuovo" possiamo andare ad inserire le proprietà e tramite "Metodi" le sue azioni.

## Proprietà

Per ogni proprietà vanno definite una serie di informazioni quali descrizione, tipo di dato della proprietà, dimensione e altro ancora.

Su ogni proprietà possiamo andare ad indicare la sua visibilità (pubblica o interna) in modo da proteggere quei dati che non devono essere manipolati dall'esterno se non dalla classe stessa.

Il tipo di dato della proprietà può essere scelto tra i tipi di dato base, oppure tra un Tipo Dato (di ambiente o applicativo) oppure tra una classe.

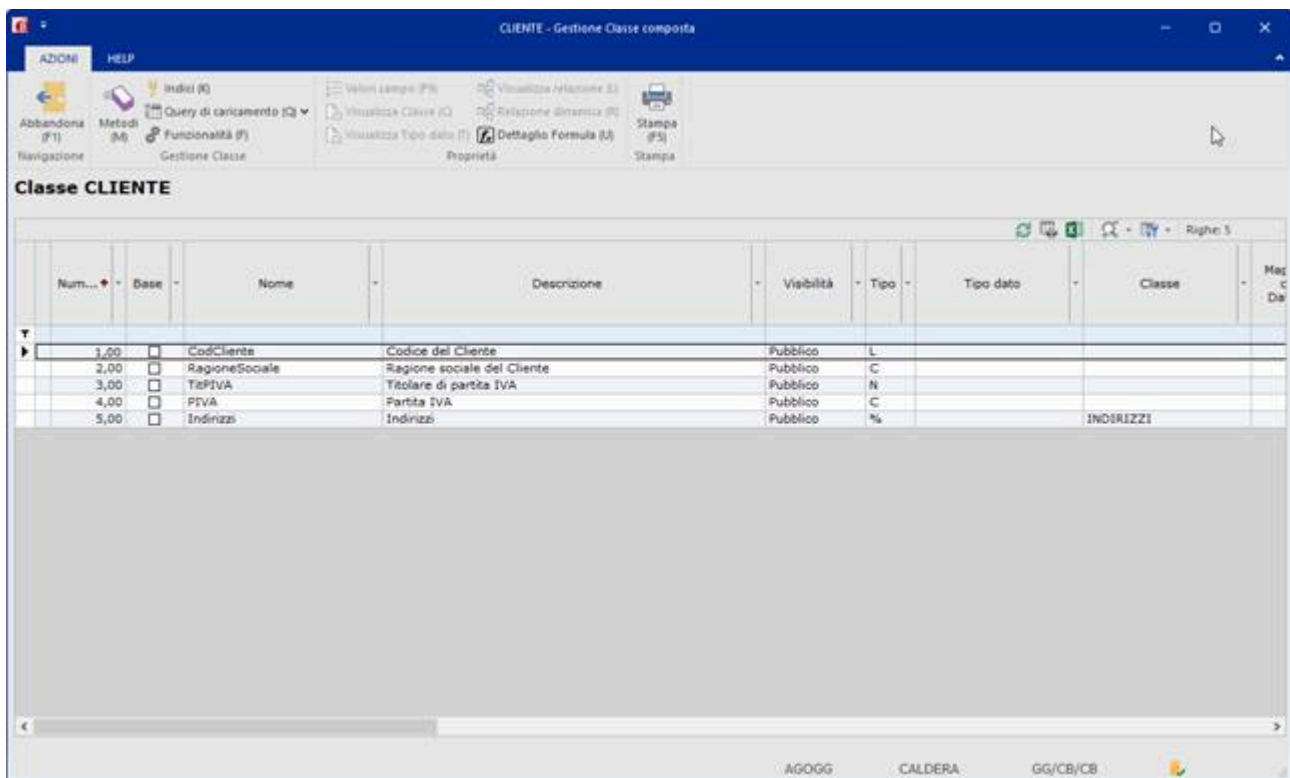
⚠ Per poter gestire il booleano è stato inserito un tipo di dato base F - Booleano. Di seguito, l'elenco con i tipi di dato gestibili:

| Elenco tipi campo |                         |                         |                                                                                             |
|-------------------|-------------------------|-------------------------|---------------------------------------------------------------------------------------------|
| Tipo              | Descrizione             | Tipo SQL                | Valori ammessi                                                                              |
| I                 | Intero                  | Smallint                | Da -32.768 a 32.767                                                                         |
| L                 | Long                    | Integer                 | Da -2.147.483.648 a 2.147.483.647                                                           |
| S                 | Singola Precisione      | Decimal                 | Precisione appross.: valori con 5 interi e 2 decimali (7 cifre totali)                      |
| D                 | Doppia Precisione       | Decimal                 | Precisione appross.: valori con 13 interi e 8 decimali                                      |
| C                 | Carattere               | Varchar                 |                                                                                             |
| N                 | Numerico (N1,N2)        | TinyInt                 | Valori numerici interi da 0 a 99                                                            |
| N                 | Numerico (N3,N4)        | Smallint                | Valori numerici interi da 0 a 9999                                                          |
| N                 | Numerico (N>4)          | Integer                 | Valori numerici interi da 0 a 2.147.483.647                                                 |
| A                 | Data                    | Datetime o Date         | Su archivio e' un campo di tipo Long in formato AAAAMMGG (Su database datetime o date)      |
| E                 | Periodo                 | Integer                 | Su archivio e' un campo di tipo Long in formato AAAAMM                                      |
| Z                 | Numerico                | -Non gestito            | Valori numerici da 0 a 61 memorizzati su 1 byte con codifica interna                        |
| R                 | Numerico                | -Non gestito            | Valori numerici con decimali ed eventuale segno in formato ASCII su archivio allineato a Sx |
| Y                 | Percentuale             | Smallint                | (Obsoleto) Utilizzato per memorizzare valori percentuali su archivio                        |
| X                 | Ascii                   | Varchar                 | Utilizzato per memorizzare il valore ASCII di un carattere                                  |
| B                 | Blob                    | Text o Varchar(Max)     | Lunghezza variabile                                                                         |
| T                 | Tipo dato               | Tipo Dato               |                                                                                             |
| Q                 | Collezione di Tipo dato | Collezione di Tipo dato |                                                                                             |
| F                 | Booleano                | Booleano                | Su archivio è un campo N1 (Può assumere i valori 0-Falso;1-Vero)                            |
| #                 | Classe                  | Classe                  |                                                                                             |
| %                 | Collezione di Classi    | Collezione di Classi    |                                                                                             |

La particolarità della classe Clienti è dovuta dagli indirizzi. L'indirizzo è una entità a sé stante e, soprattutto, può essere più di uno.

Una volta creata la classe degli Indirizzi è possibile collegarla alla classe dei Clienti utilizzando come tipologia di dato **% - collezione di classi**.

Ecco come si presenta la classe dei Clienti dopo aver caricato le proprietà che la definiscono:



## Livello di visibilità delle proprietà

In fase di definizione di una classe è possibile attribuire ad ogni singola proprietà due possibili livelli di visibilità:

- Interno
- Pubblico

Impostando il livello di visibilità come interno, nella fase di importazione di una classe in AGIS, queste proprietà non saranno considerate e quindi non potranno essere utilizzate. In questo modo si protegge la proprietà da eventuali letture e modifiche indesiderate.

Al contrario, se si necessita di mettere a disposizione una proprietà è necessario indicarla come Pubblica.

## Metodi

Per le classi esistono due tipologie differenti di metodi:

1. metodi base
2. metodi definiti dall'utente

## I metodi base

I metodi base sono tutti i metodi autogenerati dall'Ambiente di sviluppo in fase di generazione della DLL contenitore, predisposti per la manipolazione dei dati.

La sintassi per utilizzare i metodi è <Oggetto>.<Metodo>.

Esempio:

BC

```
' Pulisco il contenuto della variabile
Cliente.Clear()
```

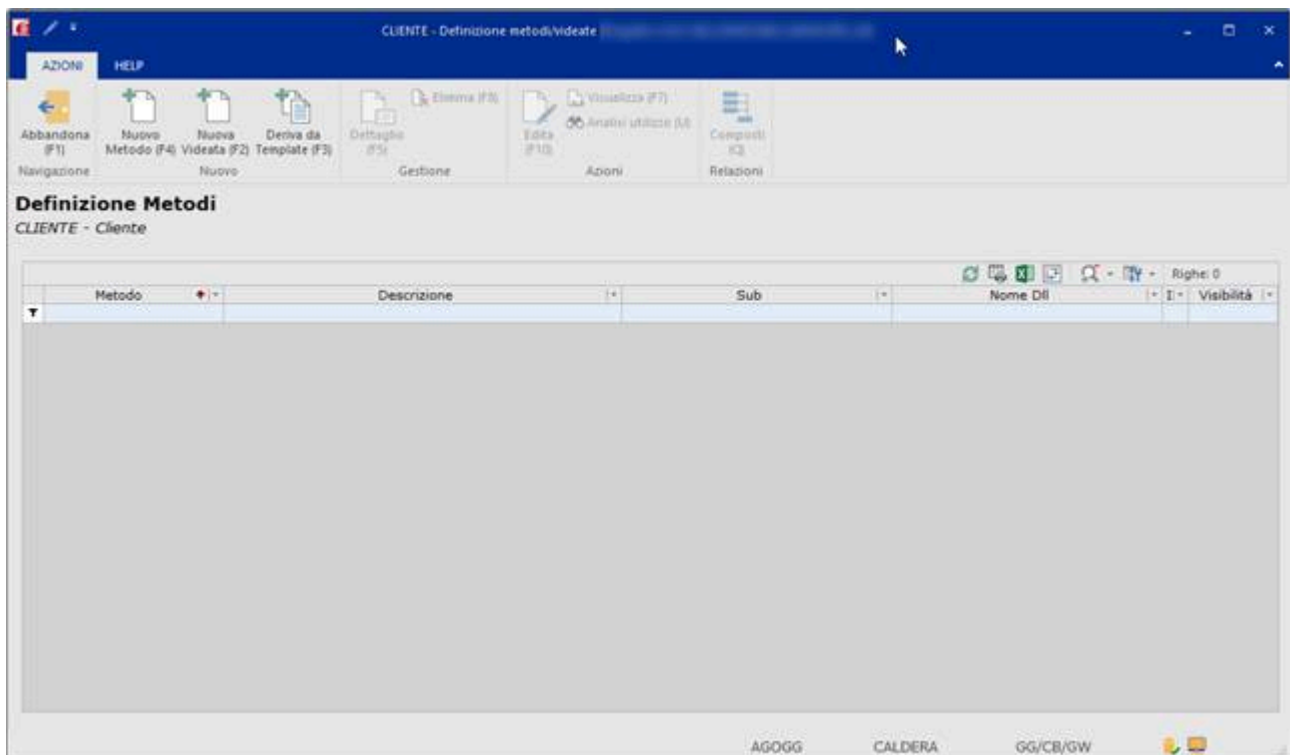


Per i metodi base disponibili è possibile fare riferimento alle schede del *Manuale di riferimento del linguaggio BC*:

- metodi base delle classi
- metodi base delle collezioni di classi

## I metodi definiti dall'utente

La loro creazione avviene tramite la gestione simile a quella sotto riportata:



Per il momento tralasciamo i bottoni "Nuova videata" e "Deriva da template" che vedremo più avanti, e vediamo nel dettaglio "Nuovo metodo".

I metodi sono dei sorgenti BC vengono creati con il nome automatico <Nome classe>\_<Nome metodo> e devono essere contenuti in DLL.

💡 Come linea guida è preferibile inserire le SUB dei metodi in DLL riferite ad un ambito applicativo di riferimento, stesso criterio utilizzato per la definizione delle normali sub.

Lo schema di riferimento è:

- *<Sigla prodotto>\_<Nome Contenitore>\_BL*, dove il suffisso BL sta per Business Logic per i metodi normali;
- *<Sigla prodotto>\_<Nome Contenitore>\_UI*, dove il suffisso UI sta per User Interface per i metodi di videata.

La creazione del metodo normale ha bisogno di un'anagrafica sul quale sono richieste le caratteristiche di base del metodo:

Una volta indicato nome e descrizione, è necessario andare a definire altre due proprietà:

- ambito di utilizzo: definisce a cui si riferisce il metodo e può valere:
  - o oggetto (si riferisce a se stesso): viene inserito come primo parametro della sub dichiarato come `This[CLASSE[<Nome classe>]]`;
  - o collezione (si riferisce ad una collezione della classe): viene inserito come primo parametro della sub dichiarato come `This[CLASSE[<Nome classe>] COLLEZIONE]`;
  - o statico (si riferisce alla classe e non alla singola oggetto istanziato): non viene definito alcun parametro sulla sub;
- visibilità: indica chi può utilizzarlo e può valere:
  - o pubblico: lo possono usare tutti, anche prodotti/progetti differenti da quello in cui viene sviluppato;
  - o interno: lo può utilizzare solo il prodotto/progetto in cui viene sviluppato;
  - o privato: può essere utilizzato solo all'interno della classe stessa, quindi nei suoi metodi.

L'opzione "Sostituzione del metodo base" lo vediamo più avanti quando affronteremo i metodi di override collegati alla base dati.

Definite queste informazioni, viene presentata la videata della sub che verrà creata e si aprirà l'editor dei sorgenti BCDevelop per permettere la stesura del codice.

Il sorgente generato in automatico prevede come unico parametro l'oggetto riferito alla classe. I parametri possono essere integrati in base alle specifiche esigenze. Al suo interno va implementata la logica applicativa specifica che il metodo deve eseguire.

Vediamo un esempio di metodo che carica una collezione:

**Dati metodo CLIENTE**

|                          |                                   |
|--------------------------|-----------------------------------|
| Metodo                   | CaricaClienti                     |
| Descrizione              | Inserimento clienti in collezione |
| Ambito di utilizzo       | Collezione                        |
| Visibilità               | Interno                           |
| Nome sub                 | CLIENTE_CARICACLIENII             |
| Sostituzione metodo base | No                                |
| Nome Dll                 | BLD_TAB_BL                        |
| Firma                    |                                   |

Buttons: OK (Invio), Abbandona (F1)

Footer: AGOGG CALDERA GG/CB/CB

In anagrafica abbiamo indicato l'ambito collezione e visibilità interna. Il sorgente BC generato è il seguente:

BC

```
'@PG CLIENTE_CARICACLIENII STRICT[1]      ' Inserimento clienti in collezione

'@SBP This[CLASSE[CLIENTE] COLLEZIONE] [INOUT]  ' Cliente

End
```

Come detto in precedenza il primo argomento è sempre un oggetto riferito alla classe per cui viene predisposto il metodo. Per convenzione la variabile viene denominata sempre This identificando l'oggetto per il quale viene eseguito il metodo.



A livello di codice BC i metodi standard vengono richiamati con la stessa sintassi di quelli base:

```
varCliente.<Metodo>().
```

Vediamo come deve essere richiamato il nostro metodo di collezione:

BC

```
' Dichiarazione di una variabile di tipo collezione
DIM varCliente[CLASSE[CLIENTE] COLLEZIONE]

' Richiamo del metodo di caricamento della collezione
varCliente.CaricaClienti()
```

Se la variabile non fosse stata definita di collezione, avrebbe dato errore in compilazione. Già l'intellisense non avrebbe proposto il metodo perché non facente parte dell'oggetto.

Come si può notare anche se la sub prevede un parametro, ovvero l'oggetto, la chiamata non lo esplicita ma viene autoreferenziato direttamente dall'oggetto per il quale deve essere eseguito il metodo.

Se il metodo prevede ulteriori parametri aggiuntivi, questi vanno indicati tra le parentesi tonde.

Quando riferiti a Oggetto o Collezione possono essere richiamati da una specifica istanza di un oggetto o di una collezione con la sintassi <Nome oggetto>.<Nome metodo>() oppure <Nome collezione>.<Nome metodo>().

Per i metodi statici invece non è necessario istanziare un oggetto ma possono essere richiamati con la sintassi <Nome classe>.<Nome Metodo>().



La sintassi per il richiamo dei metodi utilizza gli stessi formalismi utilizzati per la chiamata tramite CALL delle sub. Possono quindi essere utilizzati direttamente valori costanti o variabili preventivamente dichiarate. A differenza del richiamo dei metodi base va indicato per i parametri anche il qualificatore IN/OUT definito sul prototipo della sub di riferimento del metodo.

## Gli oggetti e il loro utilizzo in BC

La definizione delle classi in AGIS e la generazione della DLL del contenitore consentono la definizione nel codice BC di specifiche istanze riferite ad una classe. Per farlo, è necessario utilizzare la keyword *DIM[]* già vista per la dichiarazione di variabili semplice e Tipi Dato.

### Esempi

BC

```
' Sintassi
' DIM <nome oggetto>[CLASSE[<nome classe>]]

' Definizione oggetto varCliente
DIM varCliente[CLASSE[CLIENTE]]
```

Con la parola chiave *COLLEZIONE* è invece possibile istanziare una collezione/lista di oggetti dello stesso tipo:

**BC**

```
' Sintassi
' DIM <nome collezione>[CLASSE[<nome classe>] COLLEZIONE]

' Definizione oggetto varClienti
DIM varClienti[CLASSE[CLIENTE] COLLEZIONE]
```

Una volta dichiarato l'oggetto è possibile utilizzare le sue proprietà attraverso la notazione punto:

**BC**

```
' Assegnazione (SET) della proprietà CodCliente
varCliente.CodCliente = nuovoCliente

' Utilizzo (GET) proprietà RagioneSociale
DIM varStringa[STRING] = "Cliente " + varCliente.RagioneSociale.Trim()
```

 L'utilizzo della notazione punto per utilizzare nei programmi BC le proprietà dell'oggetto **non** prevede che nella sintassi sia utilizzato l'indicatore che normalmente determina il tipo di dato base.

## Operazioni di confronto

Due istanze di una classe possono essere confrontate fra loro direttamente con i semplici operatori di confronto:

**BC**

```
' Esegue una copia dell'oggetto cliente
oldCliente = varCliente

' Viene eseguito il confronto tra i due oggetti
If oldCliente <> varCliente Then ...
```

L'operazione esegue il confronto di tutte le proprietà previste dalla classe di riferimento degli oggetti. L'operazione può essere eseguita anche tra due collezioni.

## Utilizzo nella definizione dei parametri di una sub

Allo stesso modo degli altri tipi di variabile, gli oggetti e le collezioni possono essere passati come parametro ad una sub.

Definizione oggetto come parametro della sub:

BC

```
'@PG BLD_STAMPA_CLIENTE      ' Stampa Cliente

'@SBP SigoloCliente[CLASSE[CLIENTE]] [INOUT] ' Singolo cliente
```

Chiamata alla sub:

BC

```
DIM varCliente[CLASSE[CLIENTE]]

. . .

CALL BLD_STAMPA_CLIENTE(varCliente [INOUT])
```

Definizione collezione come parametro della sub:

BC

```
'@PG BLD_ELABORA_CLIENTI      ' Elabora clienti

      '@SBP ClientiDaElaborare[CLASSE[CLIENTE] COLLEZIONE] [INOUT] ' Elenco clienti
```

Chiamata alla sub:

BC

```
DIM ClientiSelezionati[CLASSE[CLIENTE] COLLEZIONE]

. . .

CALL BLD_ELABORA_CLIENTI(ClientiSelezionati [INOUT])
```

⚠ A differenza delle semplici variabili, oggetti e collezioni **non** possono essere passati per valore (parametro [IN]). Nel caso servisse preservare lo stato dell'oggetto al momento della chiamata alla sub è necessario eseguire esplicitamente la copia prima di effettuare la chiamata.

Da prestare particolare attenzione quando si trattano collezioni con un elevato numero di oggetti contenuto la cui copia può creare un'eccessiva occupazione di memoria.

## Assegnazione per valore e assegnazione per riferimento

Assegnando un oggetto (o una collezione) tramite l'operatore uguale (A=B), viene fatta *un'assegnazione per valore* e quindi il contenuto di B viene interamente copiato in A.

Questo implica una duplicazione di memoria e può essere un'operazione costosa ed esosa se si tratta di un oggetto molto grosso o di una collezione con molti elementi.

Per l'assegnazione tra oggetti/collezioni è disponibile l'operatore '=>' che effettua *un'assegnazione per riferimento* (A=>B).

In questo modo l'oggetto/collezione A *punta* all'oggetto/collezione B e di fatto ne condivide il contenuto anziché duplicarlo. Quindi una qualsiasi modifica fatta su A si rifletterà anche su B.

Per sganciare il riferimento è possibile utilizzare il metodo .Release().

L'assegnazione per riferimento è indispensabile per modificare un elemento della collezione, come vedremo più avanti, ma è anche molto utile in tanti altri contesti per non duplicare memoria.

Concludendo, possiamo dire che l'assegnazione per riferimento è da preferirsi all'assegnazione per valore, che invece andrebbe usata soltanto quando si intende preservare il contenuto dell'oggetto da copiare.

## Ricerca di un elemento in una collezione

La ricerca di un elemento all'interno di una collezione può essere eseguita utilizzando i metodi base generati in automatico nel momento in cui viene generata la DLL del contenitore delle classi.

### Esempio

Viene illustrato come ricercare il primo oggetto cliente titolare di partita IVA e con Partita IVA non valorizzata all'interno di una collezione di clienti:

BC

```
' Dichiarazione oggetto cliente titolare di partita IVA
DIM ClienteTitPIVA[CLASSE[CLIENTE]]

' Dichiarazione collezione contenente l'elenco dei clienti
DIM ElencoClienti[CLASSE[CLIENTE] COLLEZIONE]

' Ricerca primo cliente titolare di partita IVA con P.IVA non valorizzata
ClienteTitPIVA = ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="")

If ClienteTitPIVA.IsEmpty() Then ExitSr ' Oggetto non trovato
```

Il metodo `.FindFirst()` restituisce il primo oggetto della collezione che rispetta le caratteristiche indicate.

Il risultato del metodo è quindi un oggetto dello stesso tipo su cui si basa la collezione. Nell'esempio riportato sopra l'oggetto restituito dal metodo, se trovato, viene copiato nell'oggetto *ClienteTitPIVA*.

Quest'ultimo non ha nulla a che fare con quanto contenuto nella collezione. Per cui successive modifiche all'oggetto *ClienteTitPIVA* non si riflettono sulla collezione.

Con la notazione punto è anche possibile scrivere le due operazioni con un'unica istruzione:

BC

```
' Se non c'è un cliente titolare di partita IVA con P.IVA non valorizzata allora esco
If ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="").IsEmpty() Then ExitSr
```

## Modifica di un elemento di una collezione

Riprendendo l'esempio precedente vediamo ora come modificare l'oggetto della collezione restituito dal metodo `.FindFirst()`.

BC

```
' Dichiarazione oggetto cliente titolare di partita IVA
DIM ClienteTitPIVA[CLASSE[CLIENTE]]

' Dichiarazione collezione contenente l'elenco dei clienti
DIM ElencoClienti[CLASSE[CLIENTE] COLLEZIONE]

' Ricerca primo cliente titolare di partita IVA con P.IVA non valorizzata
ClienteTitPIVA => ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="")

' Modifica dell'oggetto se trovato
If Not ClienteTitPIVA.IsEmpty() Then

    ClienteTitPIVA.TitPIVA = 1
    ClienteTitPIVA.PIVA = "1234567890"

EndIf
```

Rispetto al precedente esempio cambia l'operatore di assegnazione. L'utilizzo dell'operatore "`=>`" indica che non deve essere fatta una copia dell'oggetto bensì l'oggetto *ClienteTitPIVA* diventa un **puntatore** all'oggetto estratto dal metodo `.FindFirst()`.

In questo caso quindi le successive modifiche all'oggetto si riflettono anche sulla collezione. Per scollegare l'oggetto *ClienteTitPIVA* dalla collezione sarà necessario utilizzare il metodo `.Release()`. Dopo l'esecuzione di questo metodo l'oggetto torna ad essere un'altra istanza della classe `CLIENTI` vuota.

Nell'esempio viene evidenziato anche l'utilizzo del metodo `.IsEmpty()` utilizzato per sapere se l'oggetto risultante dal metodo `.FindFirst()` non sia vuoto.

## Esecuzione azione su tutti gli elementi della collezione

Per eseguire una modifica a tutti gli elementi della collezione è possibile utilizzare il metodo `.ForEach()` come riportato qui di seguito:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, AzzerPartitaIVA(tmpCliente [INOUT]))

. . .

'@SRP AzzerPartitaIVA(tmpCliente[CLASSE[CLIENTE]] [INOUT])

    tmpCliente.PIVA = ""

Return
```

in questo caso l'oggetto *tmpCliente* viene utilizzato come puntatore alla collezione *ElencoClienti*. Questo significa che ogni modifica all'oggetto si riflette in automatico sulla collezione stessa.

La release dell'oggetto avviene in automatico alla fine del `.ForEach()`.

⚠ All'interno di un `.ForEach()` non è possibile eliminare oggetti dalla collezione. Se necessario va utilizzato un ciclo `For/Next` con metodo `.Remove()` riferito all'elemento dell'iterazione.

Il caso esposto sopra può essere semplificato nel seguente modo:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, tmpCliente.PIVA = "")
```

## Il metodo `.ForEach()` con più statement

La sintassi del metodo `.ForEach()` consente che nel suo secondo parametro siano indicati anche più di uno statement separati dal carattere ":" (due punti) come nell'esempio di seguito riportato:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, tmpCliente.PIVA="" : tmpCliente.RagioneSociale="")
```

## Utilizzo di metodi concatenati

Volendo modificare solo una parte di oggetti della collezione, si può utilizzare in modo concatenato il metodo `.Find()`:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.Find(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="").ForEach(tmpCliente, tmpCliente.PIVA =
"")
```

il metodo `.Find()` infatti restituisce una collezione di oggetti che rispettano le selezioni indicate. In questo modo solo i clienti che soddisfano l'espressione indicata saranno modificati.

## Ordinamento di una collezione

Per ordinare una collezione sono disponibili i metodi `Sort()` e `OrderBy()`.

Il primo ordina la collezione sul posto, ovvero ordina la collezione dalla quale è chiamato.

Il secondo invece non cambia l'ordinamento della collezione ma ne restituisce una nuova ordinata come richiesto (gli elementi sono gli stessi copiati per riferimento).

BC

```
ElencoClienti.Sort(CLIENTE.Nome) ' Ordina la collezione per nome cliente

ElencoClientiOrd => ElencoClienti.OrderBy(CLIENTE.Nome) ' Restituisce una
collezione di clienti ordinata per nome cliente
```

## Le classi statiche

A differenza delle classi che abbiamo visto fino ad ora, le classi statiche non rappresentano alcuna entità, per cui non possono essere istanziate per creare degli oggetti ad essa associata. Questo vuol dire che di una classe statica avrà solo metodi e nessuna proprietà.



In AGIS una classe statica si attiva andando a valorizzare l'apposito flag nell'anagrafica della classe:

**Variazione dati oggetto**

**Classi**  
Dati oggetto CLASSE\_STATICA

**Dati generali**

Nome oggetto: CLASSE\_STATICA

DFX di riferimento: [ ]

Descrizione: Esempio di classe statica

Descrizione estesa: [ ]

Categoria: [ ]

☐ Importato

Data validita' da: [ ] a [ ]

☒ **Classe statica**

OK (Invio) Abbandona (F1) Situazione (F2) Elimina (F8)

AGIS CALDERA GG/CB/CB

Non avendo proprietà, la videata successiva è quella dell'elenco dei metodi:

**CLASSE\_STATICA - Definizione metodi/Videate (Progetto AGIS: BLD\_DEMO/BLD\_DEMO/PRJ\_CB)**

**AZIONE** **HELP**

Abbandona (F1) Nuova Metodo (F4) Nuova Videata (F2) Dettaglio (F5) Elimina (F8) Edita (F10) Visualizza (F7) Analisi utilizzo (U) Composti (C2) Relazioni

**Definizione Metodi**  
CLASSE\_STATICA - Esempio di classe statica

| Metodo                   | Descrizione                  | Sub | Nome Dll                   | Visibilità                   |
|--------------------------|------------------------------|-----|----------------------------|------------------------------|
| <b>Categoria: Metodi</b> |                              |     |                            |                              |
| <b>Ambito: Statico</b>   |                              |     |                            |                              |
| NuovoMetodo              | Descrizione del nuovo metodo |     | CLASSE_STATICA_NUOVOMETODO | CLASSE_STATICA_BL<br>Interna |

AGOGG CALDERA GG/CB/CB

I metodi che vengono creati su questa classe, saranno tutti metodi statici che lavorano solo su parametri di input e quindi senza nessun legame con la classe stessa. Non dovendo istanziare alcun oggetto, il metodo verrà richiamato dalla classe stessa:

BC

```
' Richiamo di un metodo di una classe statica
CLASSE_STATICA.NuovoMetodo()
```

Nell'esempio si può vedere che non è stato istanziato nessun oggetto riferito alla classe CLASSE\_STATICA ma si sta usando direttamente la classe per sfruttare i suoi metodi.

In definitiva, si può dire che una classe statica sia un contenitore di metodi non associabili ad alcuna entità.

## Le classi di ambiente

Oltre alle classi definite dall'utente, ci sono una serie di classi definite a livello di ambiente che possono essere utilizzate nella progettazione dei programmi.

Alcuni esempi di classi di ambiente possono essere:

- BC\_VID\_ERROR: classe che permette di gestire gli errori
- BC\_DA\_VALUE: classe che permette di gestire un valore generico

Per poter utilizzare queste classi è necessario importarle nell'area di sviluppo (prodotto/progetto) di riferimento creando i relativi oggetti in AGIS tramite la funzione "Importa classe d'ambiente".